

Scala für Spark auf Databricks

Scala ist die Sprache, in der Apache Spark selbst geschrieben wurde, und bietet dadurch den direktesten und vollständigsten Zugriff auf alle Spark-Funktionen. Diese Seite beschreibt den typischen Workflow für Scala-Entwicklerinnen und -Entwickler auf Databricks – von der Notebook-Entwicklung bis zur Automatisierung über Jobs.

Grundlegender Entwicklungs-Workflow

Der übliche Ablauf für Scala-Projekte auf Databricks besteht aus drei Schritten:

1. **Code entwickeln oder importieren:** Man schreibt Scala-Code direkt in einem interaktiven Notebook oder importiert bestehenden Code.
2. **Auf einem Cluster ausführen:** Der Code wird auf einem Databricks-Cluster ausgeführt, der die eigentliche Rechenleistung bereitstellt.
3. **Erweiterte Funktionen nutzen:** Dazu zählen fortgeschrittene Datenverarbeitung, eingebaute Visualisierungen und die Automatisierung wiederkehrender Abläufe.

Apache Spark und Datenverarbeitung mit Scala

Für den Einstieg in die eigentliche Datenverarbeitung verweist die Dokumentation auf Tutorials zu DataFrames, zu Delta-Lake-Tabellen (dem Standard-Speicherformat auf Databricks) und zu Structured Streaming. Zusätzlich gibt es Links zu den offiziellen Spark-APIs für Scala, die sämtliche verfügbaren Methoden und Klassen im Detail dokumentieren.

Notebooks und Versionskontrolle

Databricks-Notebooks unterstützen Scala vollständig und bieten dabei praktische Zusatzfunktionen:

- **Eingebaute Visualisierungen:** Ergebnisse lassen sich direkt im Notebook grafisch darstellen, ohne externe Tools.
- **MLflow-Integration:** Machine-Learning-Experimente lassen sich direkt aus dem Notebook heraus nachverfolgen und verwalten.

- **Git-Ordner:** Ermöglichen die Synchronisation von Notebooks mit externen Code-Repositories (z. B. GitHub), sodass sich Scala-Code wie gewohnt versionieren lässt.

Infrastruktur und Runtime

Cluster auf Databricks lassen sich flexibel skalieren – von einem einzelnen Rechenknoten (Single Node) bis zu großen, verteilten Systemen mit vielen Knoten. Die Databricks Runtime bringt bereits vorinstallierte Bibliotheken mit, darunter Spark selbst und Delta Lake, sodass für viele Standardaufgaben keine zusätzliche Installation nötig ist.

Verschiedene Entwicklungsansätze

Scala-Code lässt sich auf mehrere Arten entwickeln:

- **Direkt in Databricks-Notebooks:** Der schnellste Weg für interaktives Arbeiten.
- **Mit externen IDEs:** Zum Beispiel IntelliJ IDEA, verbunden über Databricks Connect, für eine gewohnte Entwicklungsumgebung mit vollem IDE-Funktionsumfang.
- **Automatisiert über Jobs:** Scala-Code lässt sich als geplanter (scheduled) oder ereignisgesteuerter (triggered) Job automatisch ausführen, etwa für regelmäßige Datenpipelines.

Integration und Automatisierung

Für die Einbindung in größere Systeme und für programmgesteuerte Automatisierung stehen benutzerdefinierte Funktionen (User-Defined Functions, UDFs), SDKs sowie Kommandozeilen-Werkzeuge (CLI) zur Verfügung. Damit lässt sich Scala-Code nicht nur interaktiv, sondern auch als Teil größerer, automatisierter Systeme einsetzen.

Einordnung für Kursteilnehmer

Scala lohnt sich vor allem, wenn maximale Performance und voller Zugriff auf alle Spark-internen Funktionen gefragt sind, oder wenn ein Team ohnehin schon in einer JVM-Sprache (Java/Scala) arbeitet. Für die meisten Einsteiger im Datenanalyse-Umfeld ist PySpark der pragmatischere Startpunkt – die zugrunde liegenden Konzepte von Spark sind aber in beiden Sprachen identisch.

Quelle: <https://docs.databricks.com/aws/en/languages/scala>

Stand: 2026-08-06